

Análise de Melis Duplicadas

Mal identificado — Fulfillment (FBM)

Objetivo, modelo de dados, dicionário das tabelas e consultas

Gerado em 2026-07-14

Objetivo — melis duplicadas / "mal identificado"

O fenômeno (erro de duplicidade de meli)

1. Um produto físico **chega em 1 IS** (inbound shipment). Ex.: `70057322`.
2. Por **erro de cadastro**, o produto fica com **2 melis** (dois `INVENTORY_ID`).
3. No **put-away**, só **1 meli é fisicamente armazenada** (a correta).
4. Um **erro na tarefa de inventário** faz **subir saldo das 2 melis** no CAD **1 produto físico, 2 saldos**. A segunda é a **meli fantasma**.
5. A **fantasma** não tem lastro real na IS; sua movimentação característica na rua é um **found** (a IS dela é **inconsistente** — às vezes existe, às vezes não).
6. Um **rep de inventário** dá **LOST** na meli de origem **perda**; a fantasma fica com **saldo indevido** no CAD.
7. O **vendedor reporta** o produto como **identification** (pode ser **qualquer uma** das duas melis).
8. A **Qualidade** dá **baixa RQ** na meli fantasma, tirando-a do CAD.

Objetivo

Para os itens reportados como **identification**, trazer:

- de qual **IS (inbound)** o produto veio (origem),
- de qual **rua do MZ** ele veio,
- e o **valor** do produto,

para **quantificar e rastrear** as melis fantasma e a perda associada.

Cada linha = uma meli mal identificada, com origem (IS + rua do MZ) e valor.

Regras/mecânica (confirmadas)

- **A** — a meli reportada como **identification** pode ser **a fantasma OU a de origem** (relativo; não assumir lado).
- **B** — **pareamento das 2 melis = mesmo ADDRESS_ID** (dois `INVENTORY_ID` distintos no mesmo endereço). chave da consulta.
- **C** — **fantasma** = tem movimentação **found** na rua (IS inconsistente); **origem** = tem **inbound real** (`inbound_id` no JSON, `FBM_REASON_ENTITY='inbound'`).
- **D** — anatomia do endereço `MZ-3-072-013-05-03`:

MZ	3	072	013	05	03
área	andar	rua	prédio	nível do prédio	sub-nível

- Em SQL: `SPLIT(ADDRESS_ID, '-') [OFFSET(2)]` = rua.
- **E** — valor = `INVENTORY.REFERENCE_COST`; escopo = **TODOS os warehouses**; janela de tempo = *(a definir — ver nota de custo abaixo)*.

Custo: "todos os warehouses" + leitura do JSON da MOVEMENT tende a ficar caro (foi o que estourou 1,2 TB). Mitigação: aplicar **janela de data** na partição (`FBM_CREATED_DATE`) e só ler o JSON depois de reduzir o volume.

Tabelas por papel

Papel	Tabela / campo
Itens identification	<code>DM_SHP_FBM_QUARANT</code> (<code>REPORTED_PROBLEM_TYPE</code>)
Saldo/posição + pareamento por endereço	<code>BT_FBM_STOCK_3_ADDRESS</code> (<code>ADDRESS_ID</code> , <code>INVENTORY_ID</code>)
Found (fantasma) vs inbound (origem) + rua MZ	<code>BT_FBM_STOCK_3_MOVEMENT</code> (<code>ENTITY</code> , <code>inbound_id</code> , <code>ADDRESS_FROM</code>)

Papel	Tabela / campo
Valor / GTIN	BT_SHP_FBM_INVENTORY (REFERENCE_COST , IDENTIFIER)
Financeiro (perda paga?)	DM_FBM_FPP_HIST (FRENADO_PAGO , FBM_ISSUE_TYPE)

Caso-teste

IS 70057322 — usar para validar o resultado da consulta.

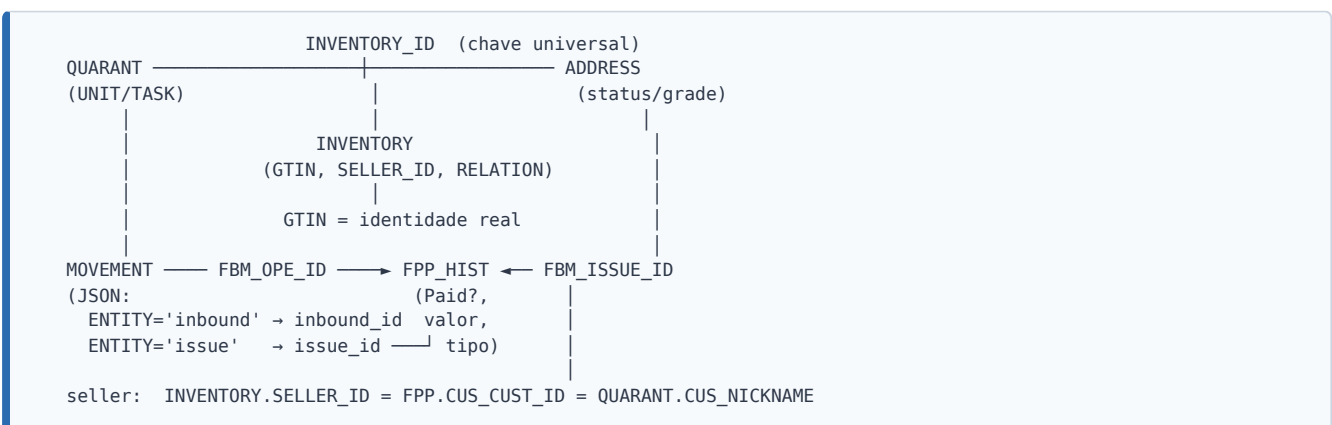
Modelo de dados — como as 5 tabelas se conectam

Visão geral do fluxo de **quarentena / mal identificado** em FBM.

Tabelas e grão

Tabela	Grão	Papel
DM_SHP_FBM_QUARANT	UNIT_ID + TASK_ID	Unidade em quarentena (problema reportado x resultante)
BT_FBM_STOCK_3_ADDRESS	INVENTORY_ID + ADDRESS_ID (+grade/lote)	Onde o estoque está agora
BT_FBM_STOCK_3_MOVEMENT	evento (FBM_OPE_ID)	Movimentações (inbound, found, transfer...)
BT_SHP_FBM_INVENTORY	INVENTORY_ID (aninhado)	Cadastro do produto (GTIN, seller, família)
DM_FBM_FPP_HIST	issue (FBM_ISSUE_ID)	Ressarcimentos por dano/perda

Grafo de junção



Chaves

- **INVENTORY_ID** — liga todas.
- **FBM_ISSUE_ID** (FPP) = **issue_id** do JSON da MOVEMENT (**ENTITY='issue'**).
- **FBM_OPE_ID** (FPP) = **FBM_OPE_ID** da MOVEMENT.
- **inbound_id** (JSON MOVEMENT **ENTITY='inbound'**) = **INBOUND_ID / SHIPMENT_ID** (FPP).
- **GTIN** (**INVENTORY.IDENTIFIER.VALUE** onde **NAME='GTIN'**) = **identidade física** do produto.
- Seller: **INVENTORY.SELLER ID** = **FPP.CUS CUST ID** = **QUARANT.CUS NICKNAME** .

Legenda de prefixos de endereço (observados)

Prefixo	Significado
QA-	Quarentena
NA-	Não alocado
MZ- / MU-	Estoque (posições de picking/storage)
DM-	Dano (destino de itens danificados)
RS-	Recebimento/reserva (a confirmar)
LT-	Perda (Definitive Lost) (a confirmar)

Abordagem recomendada (nova)

Trocar o **match frágil por descrição** por identidade e status confiáveis:

1. **Identidade** via **GTIN** (`INVENTORY.IDENTIFIER`) — não por palavras da descrição.
2. **Estado** via `ADDRESS.FBM_STOCK_STATUS/SUBSTATUS/GRADE` e `QUARANT.RESULT_PROBLEM_TYPE/SUBTYPE` (desfecho confirmado).
3. **Financeiro** via `FPP_HIST` (`FRENADO_PAGO` , `FBM_ISSUE_TYPE` , valores) ligado por `FBM_ISSUE_ID` .
4. Sempre filtrar `SITE_ID='MLB'` + galpão, e **deduplicar** (grão UNIT/issue).

Tabelas usadas na análise

Todas no dataset `meli-bi-data.WHOWNER` . São **5 tabelas físicas distintas** — a de movimentação (`BT_FBM_STOCK_3_MOVEMENT`) é lida 3 vezes, em CTEs diferentes.

#	Tabela	Papel na análise	Usada em (CTE)	Colunas usadas
1	<code>meli-bi-data.WHOWNER.DM_SHP_FBM_QUARANT</code>	Itens em quarentena	<code>QA</code>	<code>WAREHOUSE_ID</code> , <code>INVENTORY_ID</code> , <code>REPORTED_PROBLEM_TYPE</code> , <code>UNIT_CREATED_DTTM</code>
2	<code>meli-bi-data.WHOWNER.BT_FBM_STOCK_3_ADDRESS</code>	Estoque por endereço	<code>NA</code>	<code>WAREHOUSE_ID</code> , <code>INVENTORY_ID</code> , <code>ADDRESS_ID</code> , <code>FBM_AVAILABLE</code> , <code>FBM_RESERVED</code>
3	<code>meli-bi-data.WHOWNER.BT_FBM_STOCK_3_MOVEMENT</code>	Movimentações de estoque	<code>ORIGEM</code> , <code>POS_ORIGEM</code> , <code>FOUND</code>	<code>WAREHOUSE_ID</code> , <code>INVENTORY_ID</code> , <code>ADDRESS_FROM</code> , <code>ADDRESS_TO</code> , <code>FBM_CREATED_DATE</code> , <code>FBM_REASON_PROCESS</code> , <code>FBM_QUANTITY</code>
4	<code>meli-bi-data.WHOWNER.BT_SHP_FBM_INVENTORY</code>	Cadastro do produto (descrição/custo)	<code>INV</code>	<code>INVENTORY_ID</code> , <code>DESCRIPTION</code> , <code>REFERENCE_COST</code> , <code>SITE_ID</code>
5	<code>meli-bi-data.WHOWNER.DM_FBM_FPP_HIST</code>	Histórico de pagamentos (FPP)	<code>PAGOS</code>	<code>INVENTORY_ID</code> , <code>PAY_CREATED_DATETIME</code> , <code>WAREHOUSE_ID</code>

Notas

- **Prefixos:** `DM` = data mart (quarentena e pagamentos), `BT` = base/tabela transacional (endereço, movimentação, inventário).
- **A mais "trabalhada":** `BT_FBM_STOCK_3_MOVEMENT` , lida 3 vezes com filtros diferentes — origem do item (`ORIGEM`), última posição de origem (`POS_ORIGEM`) e os "achados" inbound (`FOUND`).
- **Chaves de junção:** `INVENTORY_ID` costura tudo, com apoio de `WAREHOUSE_ID` e `ADDRESS_ID` / `ADDRESS_TO` nos joins.

Dicionário — meli-bi-data.WHOWNER.DM_SHP_FBM_QUARANT

Tabela de **quarentena** (unidades com problema reportado no fulfillment). Baseado em amostra de `SELECT * ... LIMIT 1000` (2026-07-13).

Grão real = `UNIT_ID` + `TASK_ID` (unidade física + tarefa), **não** `INVENTORY_ID` .
E a amostra tem **linhas duplicadas** (mesma unidade/tarefa repetida, variando só `ITE_BASE_CURRENT_PRICE`) sempre deduplicar antes de contar.

Colunas

Identificação

Coluna	Tipo	Significado
<code>UNIT_ID</code>	id	Unidade física em quarentena (o grão de verdade)
<code>TASK_ID</code>	id	Tarefa de tratamento da unidade
<code>INVENTORY_ID</code>	id	Inventário/SKU (o que usávamos p/ join)
<code>ADDRESS_ID</code>	string	Posição — aqui é QA-... (área de quarentena), ≠ dos NA-... da tabela de estoque

Status

Coluna	Exemplos	Significado
<code>UNIT_STATUS</code>	<code>pending</code>	Situação da unidade
<code>TASK_STATUS</code>	<code>waiting_review</code>	Situação da tarefa
<code>REPORTER_USER_ID</code>	<code>2554480</code>	Quem reportou
<code>SOLVER_USER_ID</code>	<code>null</code>	Quem resolveu (<code>null</code> = ainda aberto)
<code>TASK_PROCESS_NAME</code>	<code>packing</code> , <code>picking</code> , <code>stand_alone</code>	Processo onde o problema foi detectado

Problema REPORTADO (na entrada)

Coluna	Exemplos	Significado
<code>REPORTED_PROBLEM_TYPE</code>	<code>damaged</code> , <code>identification</code>	Tipo reportado
<code>REPORTED_PROBLEM_TYPE_ID</code>	<code>1</code>	Id do tipo
<code>REPORTED_PROBLEM_DESCRIPTION</code>	<code>Product is broken</code>	Descrição livre

Problema RESULTANTE (após análise) — pode substituir suposição por fato

Coluna	Significado
<code>RESULT_PROBLEM_TYPE</code>	Tipo confirmado após tratamento (<code>null</code> enquanto não resolvido)
<code>RESULT_PROBLEM_TYPE_ID</code>	Id do tipo resultante
<code>RESULT_PROBLEM_DESCRIPTION</code>	Descrição do resultado
<code>RESULT_PROBLEM_SUBTYPE</code>	Sub-tipo do resultado
<code>PROBLEM_SUBTYPE_ID</code>	Id do sub-tipo
<code>PROBLEM_SUBTYPE_DESCRIPTION</code>	Descrição do sub-tipo

Datas

Coluna	Significado
UNIT_CREATED_DTTM / UNIT_UPDATED_DTTM	Criação/atualização da unidade em quarentena
TASK_CREATED_DTTM / TASK_UPDATED_DTTM	Criação/atualização da tarefa
UNIT_CREATED_DTTM_TZ / TASK_CREATED_DTTM_TZ	Versões com timezone (+1h na amostra)

Contexto / Produto / Comercial

Coluna	Exemplos	Significado
WAREHOUSE_ID	BRSP06	Galpão (a tabela cobre vários , não só BRSP04)
ITE_BASE_CURRENT_PRICE	6.64 , 43.24	Preço atual do item (varia entre linhas duplicadas)
CAT_CATEG_ID_L7	269718	Id de categoria (nível 7)
CAT_CATEG_NAME_L1/L2/L3	Livros / História / Brasil	Árvore de categoria
CUS_NICKNAME	REDSHOPDOBRASILOFICIAL	Seller (nickname) — enriquecimento que não tínhamos

Auditoria

Coluna	Significado
AUD_FROM_INTERFACE	Interface de origem (ex.: DM_SHP_FBM_FPP_LOST)
AUD_INS_DTTM / AUD_UPD_DTTM	Datas de carga/atualização do registro
AUD_TRANSACTION_ID	Id da transação de carga

Insights que mudam a análise

1. **Grão** = UNIT_ID , não INVENTORY_ID . Cada peça física tem seu UNIT_ID / TASK_ID . Contar/juntar por INVENTORY_ID mistura unidades diferentes do mesmo SKU.
2. **Duplicatas reais** na tabela usar SELECT DISTINCT ou QUALIFY por UNIT_ID / TASK_ID .
3. ** RESULT_PROBLEM_* e PROBLEM_SUBTYPE_* **dão o** desfecho confirmado** — dá pra parar de adivinhar com match de descrição e usar o diagnóstico oficial.
4. ADDRESS_ID **aqui é QA-...** (quarentena), diferente do NA-... do estoque — são espaços de endereço distintos.
5. CUS_NICKNAME = seller; ** CAT_* = categoria; ITE_BASE_CURRENT_PRICE ** = preço.

Dicionário — meli-bi-data.WHOWNER.BT_FBM_STOCK_3_ADDRESS

Snapshot do estoque atual por (item × endereço). Baseado em amostra `SELECT * ... LIMIT 1000` (2026-07-13). Tabela é **multi-site/multi-galpão** (amostra tem `ARBA01` / `CLRM03` = Argentina/Chile, sites `MLA` / `MLC`).

Grão ≈ (`INVENTORY_ID` , `ADDRESS_ID`), mas pode ter **mais de uma linha** por combinações de `FBM_STOCK_GRADE` / `FBM_BATCH` / `FBM_STOCK_STATUS` fonte da "inflação" que vimos (60 linhas ≈ 15 itens).

Colunas

Identificação / localização

Coluna	Exemplos	Significado
<code>INVENTORY_ID</code>	<code>SFGU94386</code>	Inventário/SKU
<code>ADDRESS_ID</code>	<code>NA-...</code> , <code>MZ-3-...</code> , <code>RS-0-...</code>	Endereço. Prefixos: NA = não alocado, MZ/MU = estoque, QA = quarentena, RS = (a confirmar)
<code>WAREHOUSE_ID</code>	<code>ARBA01</code> , <code>CLRM03</code>	Galpão
<code>SIT_SITE_ID</code>	<code>MLA</code> , <code>MLB</code> , <code>MLC</code>	Site/país (Brasil = <code>MLB</code>)
<code>ORIGINAL_INVENTORY_ID</code>	<code>null</code>	Inventário original — se populado, liga item relabelado ao original (checar em BRSP04)

Quantidades / disponibilidade

Coluna	Exemplos	Significado
<code>FBM_QUANTITY</code>	<code>1</code>	Quantidade na posição
<code>FBM_AVAILABLE</code>	<code>1 / 0</code>	Disponível (qtd/flag)
<code>FBM_RESERVED</code>	<code>0</code>	Reservado
<code>FBM_ARRIVING</code>	<code>0</code>	Em trânsito/chegando

Estado do estoque provável chave da análise

Coluna	Exemplos	Significado
<code>FBM_STOCK_STATUS</code>	<code>ok</code>	Status do estoque (checar valores p/ quarentena/bloqueado)
<code>FBM_STOCK_SUBSTATUS</code>	<code>ok</code>	Sub-status (idem)
<code>FBM_STOCK_GRADE</code>	<code>a</code>	Grade/qualidade (<code>a</code> = bom; b/c?)
<code>FBM_BATCH</code>	<code>2028-12-30</code> , <code>null</code>	Lote/validade

Datas / origem do registro

Coluna	Significado
<code>FBM_CREATED_DATE</code> / <code>FBM_LAST_UPDATED</code>	Criação / última atualização da posição
<code>FBM_CREATED_BY_APP</code>	App que criou (<code>fbm-wms-transfer</code> , <code>fbm-wms-issues</code> , <code>fbm-wms-routes-unit-handler</code>)
<code>FBM_CREATED_BY_USER</code> / <code>FBM_UPDATED_BY_USER</code>	Usuários
<code>FBM_UPDATED_BY_APP</code>	App que atualizou

Coluna	Significado
AUD_INS_DTTM / AUD_UPD_DTTM / AUD_TRANSACTION_ID	Auditoria de carga
AUD_FROM_INTERFACE	Interface de origem

Insights

1. **FBM_STOCK_STATUS** / **SUBSTATUS** / **GRADE** podem identificar estoque problemático direto — talvez sem precisar cruzar com a tabela de quarentena. Checar valores distintos.
2. **ORIGINAL_INVENTORY_ID** **existe aqui também** — na movimentação veio vazio; vale checar se nesta tabela vem populado em BRSP04 (seria o vínculo direto do relabel).
3. **Múltiplas linhas por (item, endereço)** por grade/batch/status deduplicar/agregar.
4. **Filtrar** **SIT_SITE_ID='MLB'** (e o galpão) p/ ficar só no Brasil.

Histórico de movimentações de estoque (uma linha por evento de movimento). Multi-galpão/multi-país (amostras: `MXRC02` , `BRRC01` , `BRSP04` ...).

Colunas

Coluna	Tipo	Significado
<code>FBM_OPE_ID</code>	STRING (uuid)	Id da operação/movimento
<code>WAREHOUSE_ID</code>	STRING	Galpão
<code>INVENTORY_ID</code>	STRING	Inventário/SKU movido
<code>FBM_REASON_PROCESS</code>	STRING	Processo que gerou (<code>PICK</code> , <code>lostnfound</code> , <code>stock_audit</code> , <code>cycle_count</code> , ...)
<code>FBM_REASON_ENTITY</code>	STRING	Tipo de entidade que originou — <code>inbound</code> , <code>issue</code> , ... discriminador chave
<code>FBM_REASON_ID</code>	STRING	Id da entidade (uuid ou <code>lostAndFound-<uuid></code>) — cru, 2 formatos
<code>FBM_REASON_EXTERNAL_REFERENCES</code>	JSON	Referências (ver decodificação abaixo). Coluna pesada — só ler quando necessário
<code>FBM_CREATED_DATE</code>	DATETIME	Data do movimento (provável partição)
<code>ADDRESS_FROM</code>	STRING	Origem (<code>NULL</code> = inbound/achado sem origem)
<code>ADDRESS_TO</code>	STRING	Destino (<code>NA-...</code> , <code>MU-...</code> , <code>MZ-...</code>)
<code>FBM_QUANTITY</code>	NUMERIC	Quantidade
<code>FBM_EVENT_ID</code>	STRING	Id do evento
<code>FBM_EVENT_TYPE</code>	STRING	Tipo (<code>MovementPerformed</code>)
<code>FBM_USER_ID</code>	NUMERIC	Usuário
<code>AUD_INS_DTTM</code> / <code>AUD_UPD_DTTM</code> / <code>AUD_TRANSACTION_ID</code>	—	Auditoria de carga
<code>AUD_FROM_INTERFACE</code>	STRING	Interface (<code>BQ_BT_FBM_STOCK_3_MOVEMENT</code>)
<code>DETACH_DESTINATION</code>	STRING	Destino de detach (null na amostra)
<code>FBM_ACTIONS_EXTERNAL_REFERENCES</code>	JSON	Refs de ações (null na amostra)
<code>ORIGINAL_INVENTORY_ID</code>	STRING	Inventário original (vazio nesse fluxo — 0/102525 em BRSP04NA)

Decodificação do JSON `FBM_REASON_EXTERNAL_REFERENCES`

Depende de `FBM_REASON_ENTITY` :

Quando `ENTITY = 'inbound'` (recebimento real) o "IS"

```
{
  "fiscal_source_id": "38847056",
  "fiscal_source_type": "inbound",
  "inbound_id": "38847056",
  "partiality_id": "d7235255-..."
}
```

- `inbound_id` = **Inbound Shipment (IS)** — o que a gente procurava!
`JSON_VALUE(FBM_REASON_EXTERNAL_REFERENCES, '$.inbound_id')`

Quando **ENTITY = 'issue'** (lost & found / conciliação)

```
{ "fiscal_source_type": "found", "issue_id": "1351564704",  
  "issue_uuid": "e9f3464e-...", "movement_code": "NS_FOUND", "source": "lost_and_found-v3" }
```

- **issue_id** = id do evento de achado/auditoria (NÃO é inbound).
`JSON_VALUE(FBM_REASON_EXTERNAL_REFERENCES, '$.issue_id')`

Insights

1. **O IS existe e é o **inbound_id** do JSON quando **ENTITY='inbound'**** . É um movimento **diferente** da entrada em NA (que é **ENTITY='issue'** , lost&found).
2. **Para ligar item IS:** filtrar `FBM_REASON_ENTITY='inbound'` e extrair **inbound_id** .
3. **Custo:** o JSON é caro. Filtrar por `FBM_CREATED_DATE` (partição) e por galpão antes de ler o JSON.
4. **ORIGINAL_INVENTORY_ID** continua sem serventia aqui (vazio).

Dicionário — meli-bi-data.WHOWNER.BT_SHP_FBM_INVENTORY

Cadastro do produto/inventário. Tabela **aninhada** (STRUCT/ARRAY) — vários campos são repetidos, por isso um `SELECT *` "explode" cada inventário em sub-linhas. Multi-site (amostra é `MLM` = México; Brasil = `MLB`).

Ao juntar, **selecione só escalares** (ex.: `DESCRIPTION`, `REFERENCE_COST`) para não multiplicar linhas. Para GTIN/atributos, use `UNNEST(...)` controlado.

Colunas escalares (chave)

Coluna	Exemplos	Significado
<code>INVENTORY_ID</code>	<code>K0EQ85160</code>	SKU/inventário
<code>SELLER_ID</code>	<code>623923806</code>	Seller (id numérico)
<code>ITEM_ID</code>	<code>MLM1951754091</code>	Item/anúncio no marketplace
<code>VARIATION_ID</code>	<code>null</code>	Variação do item
<code>SITE_ID</code>	<code>MLM</code> , <code>MLB</code>	Site/país
<code>TYPE</code>	<code>meli_item</code>	Tipo de inventário
<code>DESCRIPTION</code>	<code>Casco Seguridad...</code>	Título/descrição
<code>REFERENCE_COST</code>	<code>169</code>	Valor de referência do produto
<code>INSURANCE_COST</code>	<code>148.72</code>	Custo de seguro
<code>FAMILY_ID</code> / <code>FAMILY_NAME</code>	<code>2294935169610027963</code>	Família do produto (agrupa variações)
<code>USER_PRODUCT_ID</code>	<code>MLMU891923030</code>	Id do produto do usuário
<code>HAS_EXPIRATION_DATE</code>	<code>false</code>	Tem validade?
<code>OPT_IN_DATE</code> / <code>DATE_CREATED</code>	<code>—</code>	Datas de entrada/criação
<code>AUD_INS_DTTM</code> / <code>AUD_UPD_DTTM</code>	<code>—</code>	Auditoria

Campos aninhados (ARRAY/STRUCT)

Campo	Sub-campos	Significado
<code>IDENTIFIER</code>	<code>NAME</code> (<code>UPC</code> / <code>GTIN</code>), <code>VALUE</code> (<code>765619003254</code>)	Código de barras — identidade física do produto
<code>PICTURE</code>	<code>ID</code> , <code>NAME</code> , <code>URL</code> , <code>SECURE_URL</code>	Foto do produto
<code>TAG</code>	<code>sortable</code> , <code>totable</code> , <code>operates_by_pi</code> , <code>family_gm</code>	Tags de operação
<code>PRODUCT_ATTRIBUTE</code>	<code>VALUE_ID</code> , <code>VALUE_NAME</code> , <code>TYPE</code> (<code>SHIPMENT_PACKING</code>)	Atributos
<code>INVENTORY_RELATION</code>	<code>INVENTORY_ID</code> , <code>ITEM_ID</code> , <code>VARIATION_ID</code> , <code>ACTIVE</code> , <code>VERSION</code>	Relações iteminventário (possível vínculo relabel)
<code>DIMENSIONS_SET</code>	<code>WIDTH/HEIGHT/LENGTH/WEIGHT</code> + unidades	Dimensões/peso
<code>USER_PRODUCT_HISTORY</code>	<code>USER_PRODUCT_ID</code>	Histórico de produto

Insights (podem resolver o "mal identificado" de vez)

- `IDENTIFIER` (GTIN/UPC) é a identidade real do produto.** Dois `INVENTORY_ID` diferentes que são o **mesmo produto físico** tendem a compartilhar o **mesmo GTIN**. Substitui o match frágil por descrição por

um match **exato por código de barras**.

2. **INVENTORY_RELATION** pode dar o vínculo **explícito** entre inventários (mesmo produto/variação) — investigar se liga o item relabelado ao correto.
3. **FAMILY_ID** agrupa variações do mesmo produto — útil como match intermediário.
4. **SELLER_ID** aqui + **CUS_NICKNAME** na quarentena = dá pra confirmar dono.
5. Filtrar **SITE_ID='MLB'** e cuidar do aninhamento (UNNEST só quando necessário).

Dicionário — `meli-bi-data.WHOWNER.DM_FBM_FPP_HIST`

Histórico de FPP = ressarcimentos por *issues* de estoque (dano, perda, roubo, devolução). Uma linha ≈ um issue conciliado que gerou (ou não) pagamento ao seller. Tabela **multi-país** (LATAM). Fonte: `STG.FPP_HIST`.

Chaves de junção (o mais importante)

Coluna	Liga com	Uso
<code>INVENTORY_ID</code>	todas as tabelas	SKU
<code>FBM_ISSUE_ID</code>	<code>issue_id</code> do JSON da MOVEMENT (<code>ENTITY='issue'</code>)	amarra o found/lost&found ao ressarcimento
<code>FBM_OPE_ID</code>	<code>FBM_OPE_ID</code> da MOVEMENT	amarra à operação de movimento
<code>INBOUND_ID</code> / <code>SHIPMENT_ID</code>	<code>inbound_id</code> do JSON da MOVEMENT (<code>ENTITY='inbound'</code>)	recebimento (null na amostra)
<code>CUS_CUST_ID</code> / <code>CUS_NICKNAME</code>	<code>SELLER_ID</code> (INVENTORY) / <code>CUS_NICKNAME</code> (QUARANT)	seller

Issue

Coluna	Exemplos	Significado
<code>FBM_ISSUE_TYPE</code>	<code>Damaged</code> , <code>Return</code> , <code>Definitive Lost</code> , <code>Definitive Lost Cancelled</code>	Tipo do issue
<code>FBM_ISSUE_CONDITION</code>	<code>DAMAGED_BY_MELI</code> , <code>DAMAGED_BY_CARRIER</code> , <code>FRAUD_REVERSE</code> , <code>AVAILABLE</code>	Condição/causa
<code>FBM_ISSUE_STATUS</code>	<code>CONCILIATED</code>	Status
<code>FBM_PROCESS_NAME</code>	<code>Quarantine</code> , <code>Return</code> , <code>Expedition</code> , <code>Cycle count</code> , <code>Stock audit</code>	Processo de origem
<code>MOVEMENT_CODE</code> / <code>MOVEMENT_NAME</code>	<code>INV0DMGQA</code> / <code>QUARANTINE_DMG</code>	Código/nome do movimento
<code>FBM_ISSUE_DATE_CREATED</code>	—	Data de criação do issue
<code>ADDRESS_ID_FROM</code> / <code>ADDRESS_ID_TO</code>	<code>QA-...</code> <code>DM-...</code>	De/para (ex.: quarentena dano)
<code>FBM_ISSUE_QTY</code>	<code>1</code>	Quantidade
<code>LAST_ADDRESS</code>	<code>MU-TT-RC-...</code>	Último endereço conhecido

Financeiro

Coluna	Significado
<code>FRENADO_PAGO</code>	<code>Paid</code> / <code>Pending</code> — se o ressarcimento foi pago
<code>PAY_PAYMENT_ID</code> / <code>PAYMENT_ALL_ID</code>	Ids de pagamento
<code>PAY_CREATED_DATETIME</code>	Data do pagamento (era o filtro do <code>PAGOS</code>)
<code>MOV_LC_AMOUNT</code> / <code>MOV_DOL_AMOUNT</code>	Valor movido (moeda local / USD)
<code>FBM_REFERENCE_COST_LC</code> / <code>_DOL</code>	Custo de referência
<code>FBM_INSURANCE_COST_LC</code> / <code>_DOL</code>	Custo de seguro

Coluna	Significado
ADD_AMOUNT / ADD_AMOUNT_USD	Valor adicional
RECUPERO_BILLING_AMT_USD / TOTAL_AMT_USD	Recupero / total
MOV_CURRENCY_ID , USD_RATIO	Moeda / câmbio
TIPO_DE_PAGO	Tipo de pagamento

Produto / seller / classificação

Coluna	Exemplos
ITEM_TITLE	Título do produto
ITE_ITEM_DOM_DOMAIN_ID	MLM-UMBRELLAS (domínio/categoria)
CAT_CATEG_NAME_L1 / VERTICAL	Eletrodomésticos / CE
MARCA / OFS_BRAND_FANTASY_NAME	Marca
TIPOSELLER	1P / 3P
HV	HV / NO HV (high value)

Geografia / operação / auditoria

SITE_ID / SIT_SITE_ID , REGION , COUNTRY , GRUPO , FUSION , CLUSTER_OPS , CLUSTER_ICQA , WAREHOUSE_ID , FLAG_FAST_TRACK , PACKAGING_TYPE , COMMINGLING , AUD_INS_DTTM , AUD_UPD_DTTM , AUD_FROM_INTERFACE , AUD_TRANSACTION_ID .

Insights

1. **FBM_ISSUE_ID** é a ponte entre o "found" (lost&found da MOVEMENT) e o **ressarcimento**. Dá pra saber se um item achado em NA já foi **pago** como perdido.
2. **FRENADO_PAGO='Paid'** + **FBM_ISSUE_TYPE='Definitive Lost'** = item ressarcido como perdido. Se esse mesmo item **reaparece** (found em NA), é dinheiro a recuperar.
3. Filtrar por **SITE_ID='MLB'** (Brasil) e pelo galpão.
4. Substituí o **PAGOS** antigo (que só olhava **PAY_CREATED_DATETIME**) por algo bem mais rico.

Consulta principal — melis_duplicadas.sql

```
-- =====
-- MELIS DUPLICADAS / "mal identificado" (v2 – só MLB + unidades da IS)
-- =====
-- Lógica (ver sql/OBJETIVO.md):
-- 1) Itens reportados como 'identification' (QA).
-- 2) Endereços com >= 2 melis distintas COM SALDO, sendo >= 1 identification.
-- 3) Papel de cada meli: ORIGEM (tem IS/inbound) x FANTASMA (found).
-- 4) Trazer a IS de origem, as UNIDADES dentro da IS, a RUA do MZ e o VALOR.
--
-- Escopo: SOMENTE MLB (Brasil) -> STOCK/INV por site, MOVEMENT por WAREHOUSE 'BR%'.
--
-- ⚠ CUSTO: MOV_RAW lê a MOVEMENT (JSON) com janela de 365 dias e só galpões BR.
-- Ajuste `INTERVAL 365 DAY` se precisar. Faça dry-run antes.
-- ⚠ SUPosição A VALIDAR: pareamento = 2 melis com SALDO no MESMO ADDRESS_ID.
-- =====

WITH
-- 1) Itens reportados como 'identification'
QA AS (
    SELECT DISTINCT WAREHOUSE_ID, INVENTORY_ID
    FROM meli-bi-data.WHOWNER.DM_SHP_FBM_QUARANT
    WHERE REPORTED_PROBLEM_TYPE = 'identification'
),
-- 2) Saldo atual por endereço – SÓ MLB
STOCK AS (
    SELECT DISTINCT WAREHOUSE_ID, ADDRESS_ID, INVENTORY_ID
    FROM meli-bi-data.WHOWNER.BT_FBM_STOCK_3_ADDRESS
    WHERE FBM_QUANTITY > 0
    AND SIT_SITE_ID = 'MLB' -- 🇧🇷 só Brasil
),
-- Endereços-alvo: >= 2 melis e >= 1 identification
ADDR_ALVO AS (
    SELECT S.WAREHOUSE_ID, S.ADDRESS_ID
    FROM STOCK S
    LEFT JOIN QA
    ON QA.WAREHOUSE_ID = S.WAREHOUSE_ID AND QA.INVENTORY_ID = S.INVENTORY_ID
    GROUP BY 1, 2
    HAVING COUNT(DISTINCT S.INVENTORY_ID) >= 2
    AND COUNT(DISTINCT IF(QA.INVENTORY_ID IS NOT NULL, S.INVENTORY_ID, NULL)) >= 1
),
-- Melis desses endereços (o par)
MELIS AS (
    SELECT S.WAREHOUSE_ID, S.ADDRESS_ID, S.INVENTORY_ID
    FROM STOCK S
    JOIN ADDR_ALVO A USING (WAREHOUSE_ID, ADDRESS_ID)
),
-- Uma passada na MOVEMENT (só BR + janela): inbound(IS) e found
MOV_RAW AS (
    SELECT
        WAREHOUSE_ID,
        INVENTORY_ID,
        FBM_REASON_ENTITY,
        FBM_CREATED_DATE,
        JSON_VALUE(FBM_REASON_EXTERNAL_REFERENCES, '$.inbound_id') AS IS_ID
    FROM meli-bi-data.WHOWNER.BT_FBM_STOCK_3_MOVEMENT
    WHERE FBM_REASON_ENTITY IN ('inbound', 'issue')
    AND WAREHOUSE_ID LIKE 'BR%' -- 🇧🇷 só Brasil
    AND FBM_CREATED_DATE >= DATETIME_SUB(CURRENT_DATETIME(), INTERVAL 365 DAY)
),
-- Membros de cada IS (todas as melis que entraram naquele inbound)
IS_MEMBRO AS (
    SELECT DISTINCT IS_ID, INVENTORY_ID
    FROM MOV_RAW
    WHERE FBM_REASON_ENTITY = 'inbound' AND IS_ID IS NOT NULL
),
-- Total de unidades por IS (+ lista das melis, limitada)
IS_TOTAL AS (
```

```

SELECT
    IS_ID,
    COUNT(DISTINCT INVENTORY_ID) AS QTD_UNIDADES_NA_IS,
    ARRAY_TO_STRING(
        ARRAY_AGG(DISTINCT INVENTORY_ID ORDER BY INVENTORY_ID LIMIT 50), ', '
    ) AS UNIDADES_DA_IS
FROM IS_MEMBRO
GROUP BY IS_ID
),

-- Atributos por meli: teve found? qual a IS própria (última inbound)?
MELI_ATTR AS (
    SELECT
        WAREHOUSE_ID,
        INVENTORY_ID,
        LOGICAL_OR(FBM_REASON_ENTITY = 'issue') AS TEVE_FOUND,
        ARRAY_AGG(
            IF(FBM_REASON_ENTITY = 'inbound' AND IS_ID IS NOT NULL, IS_ID, NULL)
            IGNORE NULLS ORDER BY FBM_CREATED_DATE DESC LIMIT 1
        )[SAFE_OFFSET(0)] AS IS_PROPRIA
    FROM MOV_RAW
    GROUP BY 1, 2
),

-- A IS "do endereço" = a IS da meli de ORIGEM naquele endereço
IS_ENDERECO AS (
    SELECT M.WAREHOUSE_ID, M.ADDRESS_ID, MAX(A.IS_PROPRIA) AS IS_DO_ENDERECO
    FROM MELIS M
    JOIN MELI_ATTR A USING (WAREHOUSE_ID, INVENTORY_ID)
    WHERE A.IS_PROPRIA IS NOT NULL
    GROUP BY 1, 2
),

-- Cadastro (valor) – SÓ MLB, dedup por inventory
INV AS (
    SELECT INVENTORY_ID, DESCRIPTION, REFERENCE_COST
    FROM meli-bi-data.WHOWNER.BT_SHP_FBM_INVENTORY
    WHERE SITE_ID = 'MLB'
    QUALIFY ROW_NUMBER() OVER (PARTITION BY INVENTORY_ID ORDER BY AUD_UPD_DTTM DESC) = 1
)

SELECT
    M.WAREHOUSE_ID,
    M.ADDRESS_ID,
    SPLIT(M.ADDRESS_ID, '-') [SAFE_OFFSET(0)] AS AREA,
    SPLIT(M.ADDRESS_ID, '-') [SAFE_OFFSET(1)] AS ANDAR,
    SPLIT(M.ADDRESS_ID, '-') [SAFE_OFFSET(2)] AS RUA,           -- 🏠 rua do MZ
    SPLIT(M.ADDRESS_ID, '-') [SAFE_OFFSET(3)] AS PREDIO,
    M.INVENTORY_ID AS MELI,
    IF(QA.INVENTORY_ID IS NOT NULL, 'SIM', 'NÃO') AS REPORTADA_IDENTIFICATION,
    CASE
        WHEN A.IS_PROPRIA IS NOT NULL THEN 'ORIGEM (tem IS)'
        WHEN A.TEVE_FOUND THEN 'FANTASMA (found)'
        ELSE 'INDEFINIDO'
    END AS PAPEL,
    A.IS_PROPRIA,           -- IS própria da meli (null = fantasma)
    IE.IS_DO_ENDERECO,      -- IS de origem daquele endereço
    IF(MB.INVENTORY_ID IS NOT NULL, 'SIM', 'NÃO') AS MELI_PERTENCE_A_IS, -- fantasma = NÃO
    IT.QTD_UNIDADES_NA_IS,  -- 🏠 qtd de unidades dentro da IS
    IT.UNIDADES_DA_IS,      -- 🏠 as unidades dentro da IS (até 50)
    INV.DESCRPTION,
    INV.REFERENCE_COST AS VALOR
FROM MELIS M
LEFT JOIN QA
    ON QA.WAREHOUSE_ID = M.WAREHOUSE_ID AND QA.INVENTORY_ID = M.INVENTORY_ID
LEFT JOIN MELI_ATTR A
    ON A.WAREHOUSE_ID = M.WAREHOUSE_ID AND A.INVENTORY_ID = M.INVENTORY_ID
LEFT JOIN IS_ENDERECO IE
    ON IE.WAREHOUSE_ID = M.WAREHOUSE_ID AND IE.ADDRESS_ID = M.ADDRESS_ID
LEFT JOIN IS_TOTAL IT
    ON IT.IS_ID = IE.IS_DO_ENDERECO
LEFT JOIN IS_MEMBRO MB
    ON MB.IS_ID = IE.IS_DO_ENDERECO AND MB.INVENTORY_ID = M.INVENTORY_ID
LEFT JOIN INV
    ON INV.INVENTORY_ID = M.INVENTORY_ID
ORDER BY M.WAREHOUSE_ID, M.ADDRESS_ID, PAPEL;

-- =====

```

```
-- VALIDAÇÃO com a IS de exemplo (70057322) – as unidades dentro dela:
-- SELECT DISTINCT INVENTORY_ID
-- FROM meli-bi-data.WHOWNER.BT_FBM_STOCK_3_MOVEMENT
-- WHERE FBM_REASON_ENTITY = 'inbound'
--       AND JSON_VALUE(FBM_REASON_EXTERNAL_REFERENCES, '$.inbound_id') = '70057322';
-- =====
```

Consulta — quarentena_identificacao_afrouxada.sql

```
-- =====
-- Quarentena "identification" x itens achados (FOUND) em endereços NA – BRSP04
-- VERSÃO AFROUXADA (objetivo: fazer aparecer MAIS casos)
-- =====
-- 0 que a análise faz:
--   Correlaciona itens em quarentena por 'identification' que estão parados em
--   endereços NA (não alocados) com itens "achados" (inbound, sem ADDRESS_FROM)
--   que caíram no MESMO endereço NA depois, e que provavelmente são o mesmo
--   produto (heurística de similaridade de descrição).
--
-- 0 QUE MUDOU vs. a versão original (para deixar passar mais linhas):
--   (1) Janela de 180 -> 365 dias em QA, ORIGEM, POS_ORIGEM e FOUND.
--   (2) Match de descrição: limiar de >= 2 -> >= 1 palavra em comum, E com
--       NORMALIZAÇÃO (remove acentos + pontuação, casefold). Antes "cabo," e
--       "cabo", ou "lâmina" e "lamina", NÃO casavam; agora casam.
--   (3) Comparação de data: FOUND.DATA_FOUND > ... trocado por >= ...
--       (passa a incluir o achado no MESMO dia da quarentena).
--   (4) REMOVIDO o filtro (ORIGEM.ADDRESS_FROM IS NULL OR = '') que travava a
--       análise em itens de origem "inbound" e tornava inalcançáveis as
--       categorias MZ/MU/NA do próprio CASE ORIGEM_TIPO.
--
-- COMO REGULAR O QUÃO "SOLTO" FICA (botões de ajuste):
--   - Janela de tempo: troque os "INTERVAL 365 DAY".
--   - Sensibilidade do match: o ">= 1" lá no final do WHERE. Suba para >= 2
--     para menos falsos positivos; mantenha >= 1 para máxima cobertura.
--   - Para reduzir falsos positivos com >= 1, dá para adicionar uma lista de
--     stopwords no WHERE do match, ex.:
--       AND palavra_qa NOT IN ('kit','com','para','cor','preto','branco','und')
--
-- NOTA (não afeta a contagem de linhas): PAGOS entra por LEFT JOIN e só alimenta
-- o rótulo POSSUI_PAGAMENTO; a data '2026-01-01' está fixa. Se quiser o rótulo
-- coerente com a janela de análise, alinhe essa data também.
-- =====

WITH QA AS (
    SELECT
        WAREHOUSE_ID,
        INVENTORY_ID,
        REPORTED_PROBLEM_TYPE,
        DATE(UNIT_CREATED_DTTM) AS UNIT_CREATED_DTTM
    FROM meli-bi-data.WHOWNER.DM_SHP_FBM_QUARANT
    WHERE REPORTED_PROBLEM_TYPE = 'identification'
        AND UNIT_CREATED_DTTM >= DATETIME_SUB(CURRENT_DATETIME(), INTERVAL 365 DAY) -- (1) 180 -> 365
),

NA AS (
    SELECT
        WAREHOUSE_ID,
        INVENTORY_ID,
        ADDRESS_ID,
        FBM_AVAILABLE,
        FBM_RESERVED
    FROM meli-bi-data.WHOWNER.BT_FBM_STOCK_3_ADDRESS
    WHERE WAREHOUSE_ID = 'BRSP04'
        AND ADDRESS_ID LIKE 'NA%'
        AND FBM_AVAILABLE >= 0
        AND FBM_RESERVED >= 0
),

QA_NA AS (
    SELECT
        QA.INVENTORY_ID,
        QA.REPORTED_PROBLEM_TYPE,
        NA.ADDRESS_ID,
        NA.FBM_AVAILABLE,
        NA.FBM_RESERVED,
        QA.UNIT_CREATED_DTTM
    FROM NA
    JOIN QA
        ON QA.INVENTORY_ID = NA.INVENTORY_ID
        AND QA.WAREHOUSE_ID = NA.WAREHOUSE_ID
),
```

```

ORIGEM AS (
    SELECT
        INVENTORY_ID,
        ADDRESS_TO,
        ADDRESS_FROM,
        DATE(FBM_CREATED_DATE) AS ORIGEM_DATA
    FROM meli-bi-data.WHOWNER.BT_FBM_STOCK_3_MOVEMENT
    WHERE WAREHOUSE_ID      = 'BRSP04'
        AND ADDRESS_TO      LIKE 'NA%'
        AND FBM_CREATED_DATE >= DATETIME_SUB(CURRENT_DATETIME(), INTERVAL 365 DAY) -- (1) 180 -> 365
    QUALIFY ROW_NUMBER() OVER (
        PARTITION BY INVENTORY_ID, ADDRESS_TO
        ORDER BY FBM_CREATED_DATE DESC
    ) = 1
),

POS_ORIGEM AS (
    SELECT
        INVENTORY_ID,
        ADDRESS_FROM AS POSICAO_ORIGEM
    FROM meli-bi-data.WHOWNER.BT_FBM_STOCK_3_MOVEMENT
    WHERE WAREHOUSE_ID      = 'BRSP04'
        AND ADDRESS_FROM     IS NOT NULL
        AND FBM_CREATED_DATE >= DATETIME_SUB(CURRENT_DATETIME(), INTERVAL 365 DAY) -- (1) 180 -> 365
    QUALIFY ROW_NUMBER() OVER (
        PARTITION BY INVENTORY_ID
        ORDER BY FBM_CREATED_DATE DESC
    ) = 1
),

FOUND AS (
    SELECT
        WAREHOUSE_ID,
        FBM_REASON_PROCESS,
        INVENTORY_ID,
        FBM_QUANTITY,
        ADDRESS_TO,
        DATE(FBM_CREATED_DATE) AS DATA_FOUND
    FROM meli-bi-data.WHOWNER.BT_FBM_STOCK_3_MOVEMENT
    WHERE WAREHOUSE_ID      = 'BRSP04'
        AND ADDRESS_FROM     IS NULL
        AND ADDRESS_TO       LIKE 'NA%'
        AND FBM_CREATED_DATE >= DATETIME_SUB(CURRENT_DATETIME(), INTERVAL 365 DAY) -- (1) 180 -> 365
),

INV AS (
    SELECT
        INVENTORY_ID,
        DESCRIPTION,
        REFERENCE_COST -- valor do produto
    FROM meli-bi-data.WHOWNER.BT_SHP_FBM_INVENTORY
    WHERE SITE_ID = 'MLB'
),

PAGOS AS (
    SELECT DISTINCT
        INVENTORY_ID
    FROM meli-bi-data.WHOWNER.DM_FBM_FPP_HIST
    WHERE PAY_CREATED_DATETIME >= '2026-01-01' -- NOTA: data fixa; só rotula, não filtra linhas
        AND WAREHOUSE_ID      = 'BRSP04'
)

SELECT
    QA_NA.INVENTORY_ID,
    QA_NA.REPORTED_PROBLEM_TYPE,
    QA_NA.ADDRESS_ID,
    ORIGEM.ADDRESS_FROM AS ORIGEM_POSICAO,
    CASE
        WHEN ORIGEM.ADDRESS_FROM LIKE 'NA%' THEN 'Quarentena / Não alocado'
        WHEN ORIGEM.ADDRESS_FROM LIKE 'MZ%' THEN 'Posição de estoque (MZ)'
        WHEN ORIGEM.ADDRESS_FROM LIKE 'MU%' THEN 'Posição de estoque (MU)'
        WHEN ORIGEM.ADDRESS_FROM IS NULL OR ORIGEM.ADDRESS_FROM = '' THEN 'Entrada / Sem origem (inbound)'
        ELSE 'Outra origem'
    END AS ORIGEM_TIPO,
    ORIGEM.ORIGEM_DATA,
    QA_NA.FBM_AVAILABLE,
    QA_NA.FBM_RESERVED,
    QA_NA.UNIT_CREATED_DTTM,

```

```

INV_QA.DESCRPTION      AS DESCRIPTION,
INV_QA.REFERENCE_COST AS VALOR_PRODUTO,      -- valor do produto principal
STRING_AGG(
    DISTINCT CONCAT(
        FOUND.INVENTORY_ID, ' - ',
        COALESCE(INV_FOUND.DESCRPTION, ''),
        ' [valor: ', COALESCE(CAST(INV_FOUND.REFERENCE_COST AS STRING), 's/ valor'), ']',
        ' [origem: ', COALESCE(POS_FOUND.POSICAO_ORIGEM, 'sem origem'), ']'
    ),
    ', '
) AS FOUNDS,
CASE WHEN PAGOS.INVENTORY_ID IS NOT NULL THEN 'SIM' ELSE 'NÃO' END AS POSSUI_PAGAMENTO
FROM QA_NA
JOIN FOUND
    ON QA_NA.ADDRESS_ID = FOUND.ADDRESS_TO
LEFT JOIN ORIGEM
    ON ORIGEM.INVENTORY_ID = QA_NA.INVENTORY_ID
    AND ORIGEM.ADDRESS_TO = QA_NA.ADDRESS_ID
LEFT JOIN POS_ORIGEM AS POS_FOUND
    ON POS_FOUND.INVENTORY_ID = FOUND.INVENTORY_ID
LEFT JOIN INV AS INV_QA
    ON INV_QA.INVENTORY_ID = QA_NA.INVENTORY_ID
LEFT JOIN INV AS INV_FOUND
    ON INV_FOUND.INVENTORY_ID = FOUND.INVENTORY_ID
LEFT JOIN PAGOS
    ON PAGOS.INVENTORY_ID = QA_NA.INVENTORY_ID
WHERE FOUND.DATA_FOUND >= QA_NA.UNIT_CREATED_DTTM      -- (3) > trocado por >= (inclui mesmo dia)
    AND FOUND.INVENTORY_ID <> QA_NA.INVENTORY_ID
-- (4) filtro (ORIGEM.ADDRESS_FROM IS NULL OR = '') REMOVIDO de propósito
AND (
    -- (2) match normalizado (sem acento/pontuação, casefold) e limiar >= 1
    SELECT COUNT(DISTINCT palavra_qa)
    FROM UNNEST(SPLIT(
        REGEXP_REPLACE(
            LOWER(REGEXP_REPLACE(NORMALIZE(COALESCE(INV_QA.DESCRPTION, ''), NFD), r'\pM', '')),
            r'^[a-z0-9]+' , ' '
        ), ' ') AS palavra_qa
    JOIN UNNEST(SPLIT(
        REGEXP_REPLACE(
            LOWER(REGEXP_REPLACE(NORMALIZE(COALESCE(INV_FOUND.DESCRPTION, ''), NFD), r'\pM', '')),
            r'^[a-z0-9]+' , ' '
        ), ' ') AS palavra_found
        ON palavra_qa = palavra_found
    WHERE LENGTH(palavra_qa) >= 3
    ) >= 1
    ) -- (2) 2 -> 1
GROUP BY
    QA_NA.INVENTORY_ID,
    QA_NA.REPORTED_PROBLEM_TYPE,
    QA_NA.ADDRESS_ID,
    ORIGEM.ADDRESS_FROM,
    ORIGEM.ORIGEM_DATA,
    QA_NA.FBM_AVAILABLE,
    QA_NA.FBM_RESERVED,
    QA_NA.UNIT_CREATED_DTTM,
    INV_QA.DESCRPTION,
    INV_QA.REFERENCE_COST,
    PAGOS.INVENTORY_ID
ORDER BY
    QA_NA.UNIT_CREATED_DTTM DESC

```


Consulta — quarentena_identificacao_funil_diagnostico.sql

```
-- =====
-- FUNIL DE DIAGNÓSTICO – por que a query original retorna ~60 linhas?
-- =====
-- Este script NÃO retorna os casos; ele CONTA quantos "casos de quarentena"
-- (grão = INVENTORY_ID + ADDRESS_ID, o mesmo grão da saída final após o GROUP BY)
-- sobrevivem a cada etapa do funil. A etapa onde a contagem despenca é o gargalo.
--
-- ETAPAS 0..6 reproduzem a QUERY ORIGINAL (janela de 180 dias, > estrito, filtro
-- de origem e match >= 2). A etapa 6 deve bater com as suas ~60 linhas.
--
-- ETAPAS 7..10 são CENÁRIOS de afrouxamento – compare cada uma com a ETAPA 6
-- para ver quanto cada "botão" sozinho (e todos juntos) devolve de casos.
-- OBS.: os cenários usam a MESMA janela de 180 dias; a query afrouxada oficial
-- usa 365 dias e, portanto, tende a devolver ainda mais que a ETAPA 10.
--
-- Grão de contagem: COUNT(DISTINCT (QA_INV, QA_ADDR)). Uma quarentena com vários
-- FOUND conta como 1 (igual à saída final, que colapsa os founds via STRING_AGG).
-- =====

WITH QA AS (
    SELECT
        WAREHOUSE_ID,
        INVENTORY_ID,
        DATE(UNIT_CREATED_DTTM) AS UNIT_CREATED_DTTM
    FROM meli-bi-data.WHOWNER.DM_SHP_FBM_QUARANT
    WHERE REPORTED_PROBLEM_TYPE = 'identification'
        AND UNIT_CREATED_DTTM >= DATETIME_SUB(CURRENT_DATETIME(), INTERVAL 180 DAY)
),

NA AS (
    SELECT WAREHOUSE_ID, INVENTORY_ID, ADDRESS_ID
    FROM meli-bi-data.WHOWNER.BT_FBM_STOCK_3_ADDRESS
    WHERE WAREHOUSE_ID = 'BRSP04'
        AND ADDRESS_ID LIKE 'NA%'
        AND FBM_AVAILABLE >= 0
        AND FBM_RESERVED >= 0
),

QA_NA AS (
    SELECT QA.INVENTORY_ID, NA.ADDRESS_ID, QA.UNIT_CREATED_DTTM
    FROM NA
    JOIN QA
        ON QA.INVENTORY_ID = NA.INVENTORY_ID
        AND QA.WAREHOUSE_ID = NA.WAREHOUSE_ID
),

ORIGEM AS (
    SELECT INVENTORY_ID, ADDRESS_TO, ADDRESS_FROM
    FROM meli-bi-data.WHOWNER.BT_FBM_STOCK_3_MOVEMENT
    WHERE WAREHOUSE_ID = 'BRSP04'
        AND ADDRESS_TO LIKE 'NA%'
        AND FBM_CREATED_DATE >= DATETIME_SUB(CURRENT_DATETIME(), INTERVAL 180 DAY)
    QUALIFY ROW_NUMBER() OVER (
        PARTITION BY INVENTORY_ID, ADDRESS_TO
        ORDER BY FBM_CREATED_DATE DESC
    ) = 1
),

FOUND AS (
    SELECT INVENTORY_ID, ADDRESS_TO, DATE(FBM_CREATED_DATE) AS DATA_FOUND
    FROM meli-bi-data.WHOWNER.BT_FBM_STOCK_3_MOVEMENT
    WHERE WAREHOUSE_ID = 'BRSP04'
        AND ADDRESS_FROM IS NULL
        AND ADDRESS_TO LIKE 'NA%'
        AND FBM_CREATED_DATE >= DATETIME_SUB(CURRENT_DATETIME(), INTERVAL 180 DAY)
),

INV AS (
    SELECT INVENTORY_ID, DESCRIPTION
    FROM meli-bi-data.WHOWNER.BT_SHP_FBM_INVENTORY
    WHERE SITE_ID = 'MLB'
),
```

```

-- Todos os pares QA_NA x FOUND no mesmo endereço, já com origem e descrições.
BASE AS (
  SELECT
    QA_NA.INVENTORY_ID    AS QA_INV,
    QA_NA.ADDRESS_ID      AS QA_ADDR,
    QA_NA.UNIT_CREATED_DTTM,
    FOUND.INVENTORY_ID    AS FOUND_INV,
    FOUND.DATA_FOUND,
    ORIGEM.ADDRESS_FROM    AS ORIGEM_FROM,
    INV_QA.DESCRPTION      AS DESC_QA,
    INV_FOUND.DESCRPTION   AS DESC_FOUND
  FROM QA_NA
  JOIN FOUND
    ON QA_NA.ADDRESS_ID = FOUND.ADDRESS_ID
  LEFT JOIN ORIGEM
    ON ORIGEM.INVENTORY_ID = QA_NA.INVENTORY_ID
    AND ORIGEM.ADDRESS_ID = QA_NA.ADDRESS_ID
  LEFT JOIN INV AS INV_QA
    ON INV_QA.INVENTORY_ID = QA_NA.INVENTORY_ID
  LEFT JOIN INV AS INV_FOUND
    ON INV_FOUND.INVENTORY_ID = FOUND.INVENTORY_ID
),

FUNIL AS (

  -- ---- ETAPAS 0..6: reproduzem a query ORIGINAL (o "porquê das ~60") ----

  -- Universo (grão = item; referência, não é o grão dos passos seguintes).
  SELECT 0 AS ORD, '0 | QA em quarentena identification (180d)' AS ETAPA,
    (SELECT COUNT(DISTINCT INVENTORY_ID) FROM QA) AS CASOS

  UNION ALL
  SELECT 1, '1 | + parado em endereço NA (JOIN NA)',
    (SELECT COUNT(DISTINCT FORMAT('%T', STRUCT(INVENTORY_ID, ADDRESS_ID))) FROM QA_NA)

  UNION ALL
  SELECT 2, '2 | + existe FOUND no mesmo endereço NA (JOIN FOUND)',
    (SELECT COUNT(DISTINCT FORMAT('%T', STRUCT(QA_INV, QA_ADDR))) FROM BASE)

  UNION ALL
  SELECT 3, '3 | + FOUND é inventory diferente',
    (SELECT COUNT(DISTINCT FORMAT('%T', STRUCT(QA_INV, QA_ADDR)))
     FROM BASE WHERE FOUND_INV <> QA_INV)

  UNION ALL
  SELECT 4, '4 | + achado DEPOIS da quarentena (> estrito, original)',
    (SELECT COUNT(DISTINCT FORMAT('%T', STRUCT(QA_INV, QA_ADDR)))
     FROM BASE WHERE FOUND_INV <> QA_INV AND DATA_FOUND > UNIT_CREATED_DTTM)

  UNION ALL
  SELECT 5, '5 | + origem inbound (ADDRESS_FROM NULL/vazio)',
    (SELECT COUNT(DISTINCT FORMAT('%T', STRUCT(QA_INV, QA_ADDR)))
     FROM BASE
     WHERE FOUND_INV <> QA_INV
     AND DATA_FOUND > UNIT_CREATED_DTTM
     AND (ORIGEM_FROM IS NULL OR ORIGEM_FROM = ''))

  UNION ALL
  SELECT 6, '6 | + match descrição >= 2 ==> FINAL ORIGINAL (~60)',
    (SELECT COUNT(DISTINCT FORMAT('%T', STRUCT(QA_INV, QA_ADDR)))
     FROM BASE
     WHERE FOUND_INV <> QA_INV
     AND DATA_FOUND > UNIT_CREATED_DTTM
     AND (ORIGEM_FROM IS NULL OR ORIGEM_FROM = '')
     AND ( SELECT COUNT(DISTINCT p)
           FROM UNNEST(SPLIT(LOWER(DESC_QA), ' ')) AS p
           JOIN UNNEST(SPLIT(LOWER(DESC_FOUND), ' ')) AS q ON p = q
           WHERE LENGTH(p) >= 3 ) >= 2)

  -- ---- ETAPAS 7..10: CENÁRIOS de afrouxamento (compare com a ETAPA 6) ----

  UNION ALL -- só o match: >= 1 + normalizado (resto = original)
  SELECT 7, '7 | cenário: match >= 1 normalizado (resto igual ao original)',
    (SELECT COUNT(DISTINCT FORMAT('%T', STRUCT(QA_INV, QA_ADDR)))
     FROM BASE
     WHERE FOUND_INV <> QA_INV
     AND DATA_FOUND > UNIT_CREATED_DTTM
     AND (ORIGEM_FROM IS NULL OR ORIGEM_FROM = ''))

```

```

        AND ( SELECT COUNT(DISTINCT p)
              FROM UNNEST(SPLIT(REGEXP_REPLACE(LOWER(REGEXP_REPLACE(NORMALIZE(COALESCE(DESC_QA, ''), NFD),
r'\pM', '')), r'^[a-z0-9]+' , ' '), ' ') AS p
              JOIN UNNEST(SPLIT(REGEXP_REPLACE(LOWER(REGEXP_REPLACE(NORMALIZE(COALESCE(DESC_FOUND, ''), NFD),
r'\pM', '')), r'^[a-z0-9]+' , ' '), ' ') AS q ON p = q
              WHERE LENGTH(p) >= 3 ) >= 1)

UNION ALL -- só remover o filtro de origem (mantém match >= 2)
SELECT 8, '8 | cenário: SEM filtro de origem (match >= 2)',
       (SELECT COUNT(DISTINCT FORMAT('%T', STRUCT(QA_INV, QA_ADDR)))
        FROM BASE
        WHERE FOUND_INV <> QA_INV
        AND DATA_FOUND > UNIT_CREATED_DTTM
        AND ( SELECT COUNT(DISTINCT p)
              FROM UNNEST(SPLIT(LOWER(DESC_QA), ' ')) AS p
              JOIN UNNEST(SPLIT(LOWER(DESC_FOUND), ' ')) AS q ON p = q
              WHERE LENGTH(p) >= 3 ) >= 2)

UNION ALL -- só trocar > por >= na data (mantém resto do original)
SELECT 9, '9 | cenário: DATA_FOUND >= (inclui mesmo dia)',
       (SELECT COUNT(DISTINCT FORMAT('%T', STRUCT(QA_INV, QA_ADDR)))
        FROM BASE
        WHERE FOUND_INV <> QA_INV
        AND DATA_FOUND >= UNIT_CREATED_DTTM
        AND (ORIGEM_FROM IS NULL OR ORIGEM_FROM = '' )
        AND ( SELECT COUNT(DISTINCT p)
              FROM UNNEST(SPLIT(LOWER(DESC_QA), ' ')) AS p
              JOIN UNNEST(SPLIT(LOWER(DESC_FOUND), ' ')) AS q ON p = q
              WHERE LENGTH(p) >= 3 ) >= 2)

UNION ALL -- TODOS juntos (ainda 180d; a query .sql usa 365d e devolve mais)
SELECT 10, '10 | cenário: TODOS juntos (match>=1 norm + sem origem + data >=) [180d]',
       (SELECT COUNT(DISTINCT FORMAT('%T', STRUCT(QA_INV, QA_ADDR)))
        FROM BASE
        WHERE FOUND_INV <> QA_INV
        AND DATA_FOUND >= UNIT_CREATED_DTTM
        AND ( SELECT COUNT(DISTINCT p)
              FROM UNNEST(SPLIT(REGEXP_REPLACE(LOWER(REGEXP_REPLACE(NORMALIZE(COALESCE(DESC_QA, ''), NFD),
r'\pM', '')), r'^[a-z0-9]+' , ' '), ' ') AS p
              JOIN UNNEST(SPLIT(REGEXP_REPLACE(LOWER(REGEXP_REPLACE(NORMALIZE(COALESCE(DESC_FOUND, ''), NFD),
r'\pM', '')), r'^[a-z0-9]+' , ' '), ' ') AS q ON p = q
              WHERE LENGTH(p) >= 3 ) >= 1)
       )

SELECT
  ORD,
  ETAPA,
  CASOS,
  ROUND(100 * SAFE_DIVIDE(CASOS, FIRST_VALUE(CASOS) OVER (ORDER BY ORD)), 1) AS PCT_DO_UNIVERSO
FROM FUNIL
ORDER BY ORD;

```